# Spaldotech Operations — Technic Solder (api.spaldotechoperations.co.uk) Build Guide

Oct 18, 2026 · @Aidan Spalding

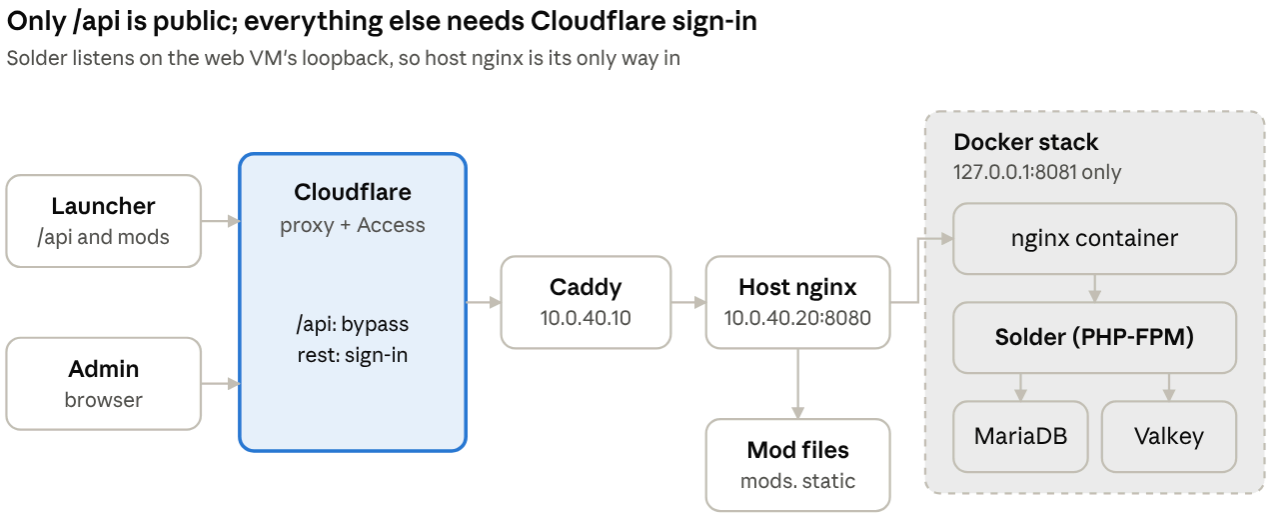
Technic Solder runs in Docker on the web VM (10.0.40.20) and is published as api.spaldotechoperations.co.uk. The Technic Launcher reaches its `/api` freely, while the admin panel sits behind Cloudflare Access and Solder's own login. Mod files are served from a separate static site, mods.spaldotechoperations.co.uk.

| Field | Value |
|---|---|
| Document ID | SPO-HL-SLD-001 |
| Version | 1.0 (as built) |
| Status | Implemented 18 October 2026 |
| Owner | Spaldotech Operations |
| Classification | Internal / Technical |
| Related standards | SPO-HL-PRX-001 (Caddy proxy), SPO-HL-SEC-001, SPO-HL-SIEM-001 |

**Scope:** Docker on the web VM, the Solder stack, the host nginx sites for `api` and `mods`, the Caddy blocks, Cloudflare DNS and Access. The proxy, firewall and DMZ design are covered by SPO-HL-PRX-001 and not repeated here.

## 1. Design

Solder runs as a Docker stack on the existing web VM, behind the same Cloudflare, Caddy and host-nginx chain as the landing page. Cloudflare Access splits the hostname: the launcher's `/api` paths are public, and every other path needs a Cloudflare sign-in before Solder's own login.




request path · launcher and admin to the Solder containers

Each layer keeps one job:

- **Cloudflare** hides the origin, applies its WAF, and enforces Access sign-in for the admin panel.
- **Caddy** terminates TLS with a Let's Encrypt certificate and accepts traffic from Cloudflare only.
- **Host nginx** adds the security headers, logs the real visitor, blocks dotfile probes, and is the only route to the container.
- **The Docker stack** publishes its nginx on `127.0.0.1:8081` only. PHP-FPM, MariaDB and Valkey sit on Docker's internal network and are never reachable from outside the VM.

Mod files are served separately as static files from mods.spaldotechoperations.co.uk, which the launcher downloads directly. That keeps large downloads off the PHP application.

## 2. Build parameters

| Parameter | Value |
|---|---|
| Web VM | ops-webserver-01, 10.0.40.20 (DMZ, VLAN 40) |
| Proxy | core-proxy-01, 10.0.40.10 |
| Solder install folder | `/opt/solder` (outside any web root) |
| Solder containers | nginx, solder (PHP-FPM), mysql (MariaDB), redis (Valkey) |
| Container port | `127.0.0.1:8081` on the web VM, loopback only |

| Parameter | Value |
|---|---|
| Host nginx listener | `10.0.40.20:8080` |
| Admin and API hostname | api.spaldotechoperations.co.uk |
| Launcher API URL | `https://api.spaldotechoperations.co.uk/api/` |
| Mod files hostname | mods.spaldotechoperations.co.uk |
| Mod files web root | `/var/www/mods.spaldotechoperations.co.uk/html` |
| Database data | `/opt/solder/docker/mysql` |
| nginx logs | `/var/log/nginx/domains/<hostname>/` |
| Cloudflare Access team | spaldotech.cloudflareaccess.com |

### Ports

| Hop | From | To | Port |
|---|---|---|---|
| Internet to Cloudflare | Visitor or launcher | Cloudflare edge | TCP 443 |
| Cloudflare to proxy | Cloudflare ranges | 10.0.40.10 (Caddy) | TCP 443, via forward F1 |
| Proxy to web VM | 10.0.40.10 | 10.0.40.20 (host nginx) | TCP 8080 |
| Host nginx to Solder | 127.0.0.1 | 127.0.0.1 (Solder's nginx container) | TCP 8081 |
| Inside Docker | Solder's nginx | PHP-FPM, MariaDB, Valkey | Docker internal network only |

## 3. Install Docker on the web VM

Docker comes from Docker's own repository. Debian 13 no longer ships `software-properties-common` (`add-apt-repository`), so the repository is added by hand with a signed keyring.

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o
/etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-
by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/debian $(.
/etc/os-release && echo "$VERSION_CODENAME") stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list
sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io
docker-compose-plugin
```

Two behaviours to plan around:

- **Published ports bypass the host firewall.** Docker writes its own rules ahead of nftables, so a port published on all addresses is reachable even if nftables blocks it. Every container port on this VM is therefore published on `127.0.0.1` only.
- **Restarting nftables can break container networking.** If `/etc/nftables.conf` begins with `flush ruleset`, a restart also removes Docker's rules. Run `sudo systemctl restart docker` after any nftables restart.

# 4. Install Solder

The steps below are in the order that avoids every first-build failure in §9: the settings, permissions and frontend assets are all in place before the first start.

## 4.1 Get the code

```
sudo git clone https://github.com/TechnicPack/TechnicSolder.git /opt/solder
cd /opt/solder
sudo cp .env.example .env
```

## 4.2 Configure `.env`

`.env.example` carries Laravel's generic database placeholders (`root`, `laravel`). Replace them before the first start: MariaDB creates its database and user only once, on first start with an empty data folder.

Generate the application key and two passwords:

```
echo "base64:$(openssl rand -base64 32)"   # APP_KEY
openssl rand -base64 24                    # DB_PASSWORD
openssl rand -base64 24                    # SOLDER_INITIAL_ADMIN_PASSWORD
```

Store all three in the password manager, then `sudoedit .env` and set:

```
APP_URL=https://api.spaldotechoperations.co.uk
APP_KEY=base64:<generated value>

SOLDER_PORT=127.0.0.1:8081

DB_CONNECTION=mysql
DB_HOST=mysql
DB_PORT=3306
DB_DATABASE=solder
DB_USERNAME=solder
DB_PASSWORD=<generated password>

SOLDER_INITIAL_ADMIN_EMAIL=aidan.spalding@spaldotechoperations.co.uk
SOLDER_INITIAL_ADMIN_PASSWORD=<generated password>

SOLDER_REPO_LOCATION=https://mods.spaldotechoperations.co.uk/
SOLDER_MIRROR_URL=https://mods.spaldotechoperations.co.uk/
```

- `SOLDER_PORT=127.0.0.1:8081` makes Docker publish Solder's nginx on loopback only (§3).
- Never change `APP_KEY` once Solder is in use; sessions and encrypted settings become unreadable.
- The database user is never `root`: the app gets its own account limited to its own database.

### 4.3 Permissions

The project folder is mounted into the containers, where Solder runs as uid 33 (the same as `www-data` on Debian), not root:

```
cd /opt/solder
sudo chown root:33 .env && sudo chmod 640 .env
sudo chown -R 33:33 storage bootstrap/cache
```

`.env` stays root-owned and hidden from other local users, but the app can read it. Do not use `chmod 600`; the app then cannot read its own settings.

### 4.4 Build the frontend assets

A throwaway Node container builds them, so Node never needs installing on the VM:

```
sudo docker run --rm -v /opt/solder:/app -w /app node:22-alpine sh -c "npm ci
&& npm run build"
ls public/build/            # manifest.json and an assets folder
sudo rm -rf node_modules    # build-time only
```

### 4.5 First start

```
sudo docker compose up -d
sudo docker compose ps                    # all four Up; solder must not show
"Restarting"
sudo ss -lntp | grep 8081                 # 127.0.0.1:8081 only, never
0.0.0.0
curl -sI http://127.0.0.1:8081/login | head -1
```

The first start takes a minute or two while the database is created and migrated.

### 4.6 Admin account

Sign in at https://api.spaldotechoperations.co.uk once the rest of the chain is in place (§5–§7). Then delete the `SOLDER_INITIAL_ADMIN_PASSWORD` line from `.env`, so the password is not left in plain text.

To reset a lost admin password, without the password reaching the shell history:

```
cd /opt/solder
read -rs -p 'New Solder admin password: ' PW; echo
sudo docker compose exec -e NEWPW="$PW" solder php artisan tinker --execute='
$u = App\Models\User::where("email",
"aidan.spalding@spaldotechoperations.co.uk")->first();
$u->password = Hash::make(getenv("NEWPW"));
$u->save();
echo "Password updated for {$u->email}\n";'
unset PW
```

## 5. Host nginx

The host nginx on 10.0.40.20:8080 is the single entry point on the web VM. It adds security headers, logs the real visitor, refuses unknown hostnames, and proxies `api` to the Solder container.

### 5.1 Security header files

Three files in `/etc/nginx/global-security/` separate the headers every site shares from each site's Content-Security-Policy:

| File | Contents | Used by |
|---|---|---|
| `security-headers.conf` | COOP, CORP, nosniff, X-Frame-Options, Referrer-Policy, Permissions-Policy. No CSP | Every site |
| `csp-static-landing.conf` | Strict CSP: no scripts, forms or framing | www (landing page) |
| `csp-web-app.conf` | Light CSP: no framing, plugins or `<base>` hijacking | api (Solder) |

Include exactly one CSP file per site. Check that each site sends one CSP header with `curl -sI … | grep -i content-security`.

## 5.2 The api site

`/etc/nginx/sites-available/api.spaldotechoperations.co.uk`:

```
server {
    listen 10.0.40.20:8080;
    server_name api.spaldotechoperations.co.uk;
    charset utf-8;
    server_tokens off;
    client_max_body_size 100m;          # room for mod uploads through the
admin panel

    include global-security/security-headers.conf;
    include global-security/csp-web-app.conf;

    # Log the real visitor, not the proxy (Caddy sends X-Real-IP)
    set_real_ip_from 10.0.40.10;
    real_ip_header X-Real-IP;

    # Never pass dotfile probes (.env, .git) through to the app
    location ~ /\.(?!well-known/) {
        deny all;
    }

    location / {
        limit_except GET POST PUT PATCH DELETE OPTIONS { deny all; }

        proxy_pass http://127.0.0.1:8081;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $remote_addr;
```

```
        proxy_set_header X-Forwarded-Proto https;
        proxy_set_header X-Forwarded-Host $host;

        # Stop duplicates if Solder sets these itself; ours win
        proxy_hide_header X-Frame-Options;
        proxy_hide_header X-Content-Type-Options;
        proxy_hide_header Referrer-Policy;
    }

    access_log
/var/log/nginx/domains/api.spaldotechoperations.co.uk/access.log;
    error_log
/var/log/nginx/domains/api.spaldotechoperations.co.uk/error.log;
}
```

`X-Forwarded-Proto` is fixed to `https`, because visitors always arrive over HTTPS through Caddy, and Solder needs to know it to build correct links.

```
sudo install -d -m 0755 -o root -g adm
/var/log/nginx/domains/api.spaldotechoperations.co.uk
sudo ln -s /etc/nginx/sites-available/api.spaldotechoperations.co.uk
/etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

Only one catch-all `default_server` may exist per address. Never copy another site's file to start this one, or the catch-all comes with it; check with `grep -rn default_server /etc/nginx/sites-enabled/`, which must list exactly one line.

## 5.3 The mods site

Create it with `sudo ./new-nginx-site.sh`: hostname `mods.spaldotechoperations.co.uk`, profile read-only, no PHP. Upload mod files into its web root in Technic's layout:

```
mods/<slug>/<slug>-<version>.zip
```

Files stay owned by root and readable by nginx's group: `chown root:www-data`, mode `640` for files and `750` for folders.

# 6. Caddy and Cloudflare DNS

## 6.1 Caddy blocks on core-proxy-01

Add both to `/etc/caddy/Caddyfile`. They use the snippets from SPO-HL-PRX-001 §7 and have no health checks: with one backend, a failed check only turns every request into a 503.

```
# Technic Solder (admin panel and launcher API) on the web VM
api.spaldotechoperations.co.uk {
        import cf_tls
        import secure_headers
        import access_log
        reverse_proxy 10.0.40.20:8080 {
                header_up X-Real-IP {client_ip}
                header_down -Via
        }
}

# Mod files for the Technic Launcher
mods.spaldotechoperations.co.uk {
        import cf_tls
        import secure_headers
        import access_log
        reverse_proxy 10.0.40.20:8080 {
                header_up X-Real-IP {client_ip}
                header_down -Via
        }
}
```

```
sudo caddy fmt --overwrite /etc/caddy/Caddyfile
sudo -u caddy -H bash -c 'set -a; . /etc/caddy/caddy.env; /usr/bin/caddy
validate --config /etc/caddy/Caddyfile'
sudo systemctl reload caddy
```

If a block ever does use health checks, add `health_headers { Host <hostname> }`. Without it Caddy checks with the backend's IP as the hostname, nginx's catch-all refuses it, and the site goes down.

## 6.2 Cloudflare DNS

| Name | Type | Content | Proxy status |
|---|---|---|---|
| api | CNAME | spaldotechoperations.co.uk | Proxied |

| Name | Type | Content | Proxy status |
|---|---|---|---|
| mods | CNAME | spaldotechoperations.co.uk | Proxied |

Caddy issues each certificate through the DNS-01 challenge once the record exists. Until it has, Cloudflare returns 525. Confirm issuance with `journalctl -u caddy | grep -i 'certificate obtained'`.

Expect scanners within minutes. Every new certificate is published in Certificate Transparency logs, which scanners watch; the first probes for `/.env` and `/.git/HEAD` reached api within minutes of its certificate. Set up §7 before announcing the hostname.

## 7. Cloudflare Access

Two Access applications cover the same hostname. When rules overlap on a common root path, Cloudflare applies the more specific path (app paths), so `/api` is bypassed and everything else requires sign-in.

| Application | Public hostnames (subdomain, domain, path) | Policy | Result |
|---|---|---|---|
| Solder admin | `api`, `spaldotechoperations.co.uk`, path empty | `Admins only`: Allow, Include Emails = your address | Sign-in for every page |
| Solder launcher API | `api` with paths `api`, `api/` and `api/*` | `Public launcher API`: Bypass, Include Everyone | Launcher reaches the API without sign-in |

### 7.1 One-time setup

1. Open **Zero Trust** in the Cloudflare dashboard. Team name `spaldotech`, plan Free (up to 50 users).
2. Under authentication, add **One-time PIN** as a login method. Access then emails a code instead of using a password.

### 7.2 Create the applications

For each application: **Access controls → Applications → Create new application → Self-hosted and private**, then fill in the table above.

- Path fields have no leading slash: `api`, not `/api`. Cloudflare adds it.

- `api/*` does not match `/api` or `/api/` themselves, which is why all three paths are listed.
- Bypass can't check identity, so its rule is Everyone. Bypassed requests are not logged by Access (policies); Caddy and nginx still log them.
- Click through every step of the wizard until the application appears in the list. An application abandoned before the final step is never created, and Access then applies the admin policy to `/api` too.

### 7.3 Why it can't be bypassed

Access only protects traffic that passes through Cloudflare. OPNsense forwards 443 only from Cloudflare's ranges, and the proxy's nftables enforces the same list (SPO-HL-PRX-001 §10–§11), so nobody can reach Solder directly by the home IP and skip the sign-in.

## 8. Verification tests

Run the Access tests with curl rather than a browser: a browser that is already signed in hides whether the bypass works.

| ID | Test | How | Expected |
|---|---|---|---|
| S01 | Containers healthy | `sudo docker compose ps` in `/opt/solder` | All four Up; solder not Restarting |
| S02 | Loopback only | `sudo ss -lntp \| grep 8081` on the web VM | `127.0.0.1:8081` only |
| S03 | Solder answers locally | `curl -sI http://127.0.0.1:8081/login` | 200 |
| S04 | Host nginx proxies api | From the proxy: `curl -sI -H 'Host: api.spaldotechoperations.co.uk' http://10.0.40.20:8080/login` | 200, one CSP header |
| S05 | Links use HTTPS | From the proxy: fetch `/login` as in S04 and look at the `href`, `src` and `action` links | All start `https://api.spaldotechoperations.co.uk` |
| S06 | Admin needs sign-in | `curl -s -o /dev/null -w '%{http_code} %{redirect_url}\n' https://api.spaldotechoperations.co.uk/login` | 302 to spaldotech.cloudflareaccess.com |
| S07 | API bypasses sign-in | The same command for `/api`, `/api/` and `/api/modpack` | 200 each, no redirect |
| S08 | Full admin sign-in | Private browser window to https://api.spaldotechoperations.co.uk | Cloudflare email code, then Solder's login, then the dashboard |
| S09 | Dotfiles blocked | `curl -s -o /dev/null -w '%{http_code}\n' https://api.spaldotechoperations.co.uk/api/.env` | 403 |
| S10 | Mod files served | Fetch a known zip from https://mods.spaldotechoperations.co.uk/mods/… | 200 and the file downloads |

| ID | Test | How | Expected |
|---|---|---|---|
| S11 | Launcher works | Technic Platform modpack Solder URL set to `https://api.spaldotechoperations.co.uk/api/`; install the pack in the launcher | Pack and mods download |
| S12 | Survives reboot | Reboot the web VM, then S01–S04 | All pass; Docker restarts the stack itself |

# 9. Troubleshooting

Every failure met during the first build, in the order the request path meets them. Work from the outside in: Cloudflare, then Caddy, then host nginx, then the containers.

| Symptom | Where | Cause | Fix |
|---|---|---|---|
| Sign-in demanded on `/api` | Cloudflare | The launcher API application was never created (wizard not finished), or its paths are wrong | Check S07; confirm the application is listed, with paths `api`, `api/`, `api/*` and a Bypass policy. Each redirect's `kid=` shows which application caught it |
| 525 SSL handshake failed | Cloudflare to Caddy | Caddy has no certificate for the hostname: block missing, reload failed, or issuance not finished | Add the block, validate as the caddy user, reload, and wait for "certificate obtained" |
| Certificate messages missing from the journal | Caddy | A global `log` block sends Caddy's own messages to `/var/log/caddy/caddy.log` | Read that file, or remove the global block (SPO-HL-PRX-001 §7) |
| 503 in under 1 ms, size 0 | Caddy | Active health checks failing, so the only backend is marked down. Without `health_headers`, checks use the backend IP as the hostname and hit nginx's catch-all | Remove the health lines, or add `health_headers { Host <hostname> }` |

| Symptom | Where | Cause | Fix |
|---|---|---|---|
| 502 with `Server: Caddy` | Caddy to nginx | nginx closed the connection: the site isn't loaded, so the catch-all answered 444 | `sudo nginx -t`, fix, reload; `sudo nginx -T \| grep server_name` shows loaded sites |
| `duplicate default server for 10.0.40.20:8080` | nginx | Two catch-all blocks, often from copying another site's file | Keep one; `grep -rn default_server /etc/nginx/sites-enabled/` must list one line |
| `invalid number of arguments in "include"` | nginx | Missing `;` on the line above, or two files in one include | One file per include, each ending in `;` |
| 502 with `Server: nginx` | nginx to container | Nothing healthy on 127.0.0.1:8081 | Check the site's error log for the upstream error, then `sudo docker compose ps` |
| solder container `Restarting (1)` | Container | The app exits at start-up; the reason is in its log | `sudo docker compose logs --tail 60 solder` |
| `Access denied for user 'root'@…` | Container | `.env` still has Laravel's placeholders (`root`, `laravel`), not the credentials MariaDB was created with | Correct `DB_*` in `.env`. On a fresh install only, reset the empty database (§9.1) |
| `public/build/ is missing` | Container | Frontend assets never built | Build them with the Node container (§4.4) |
| 500, log says `No application encryption key` | Container | `APP_KEY` empty | Generate a key (§4.2), put it in `.env`, `docker compose up -d`, then `php artisan config:clear` |
| Can't read `.env`, or `storage` permission denied | Container | Files root-only after `chmod 600` or a root clone | Apply §4.3 |

| Symptom | Where | Cause | Fix |
|---|---|---|---|
| Generated admin password not in the logs | Container | `docker compose up -d` after a `.env` change recreates the container and its log starts empty | Reset the password with tinker (§4.6) |

### 9.1 Resetting an empty database

Only while Solder holds no data, for example when the first start used the wrong credentials:

```
cd /opt/solder
sudo docker compose down
sudo find /opt/solder/docker/mysql -mindepth 1 -delete
sudo docker compose up -d
```

Never run this on a working install: it deletes the database. To change the database password later, change it inside MariaDB first, then in `.env`.

## 10. Operations

### 10.1 Updating Solder

1. Snapshot the web VM in Proxmox and take a database dump (§10.2).
2. Pull the new version and rebuild:

```
cd /opt/solder
sudo git pull
sudo docker run --rm -v /opt/solder:/app -w /app node:22-alpine sh -c "npm ci
&& npm run build"
sudo rm -rf node_modules
sudo chown -R 33:33 storage bootstrap/cache      # new files from git arrive
root-owned
sudo docker compose up -d --build
sudo docker compose exec solder php artisan config:clear
sudo docker compose exec solder php artisan migrate:status
```

3. Run S01–S07 and S11.

### 10.2 Backups

- **VM backup:** the web VM's PBS job covers `/opt/solder` (code, `.env` and database files) and the mods web root.

- **Database dump:** a dump is consistent even while Solder is running, unlike the raw database files. Take one before every update, and nightly:

```
cd /opt/solder
sudo bash -c 'umask 077; set -a; . ./.env; set +a
  docker compose exec -T mysql mariadb-dump -u"$DB_USERNAME" -p"$DB_PASSWORD"
"$DB_DATABASE" \
  | gzip > /root/solder-db-$(date +%F).sql.gz'
```

- **Secrets:** `APP_KEY`, `DB_PASSWORD` and the admin password live in the password manager. Without `APP_KEY`, a restored database cannot decrypt its own data.

## 10.3 Monitoring

When core-siem-01 is built (SPO-HL-SIEM-001), give the web VM a Wazuh group that adds:

- the nginx logs, with one wildcard for every site: `/var/log/nginx/domains/*/access.log`;
- file monitoring on `/opt/solder/compose.yml` and `/etc/nginx`, plus `/opt/solder/.env` with `nodiff`, so a change alerts without the secrets being stored.

## 10.4 Sign-off checklist

- [ ] Docker installed; Solder's port published on 127.0.0.1 only (S02).
- [ ] `.env` complete: APP_URL, APP_KEY, SOLDER_PORT, dedicated database user, repository URLs.
- [ ] Secrets stored in the password manager; initial admin password removed from `.env`.
- [ ] `.env` root:33 640; `storage` and `bootstrap/cache` owned by 33.
- [ ] Frontend assets built.
- [ ] Host nginx api and mods sites live, one catch-all, one CSP each.
- [ ] Caddy blocks and Cloudflare records in place; certificates issued.
- [ ] Both Access applications created; S06 and S07 pass.
- [ ] Admin email changed from the default; Solder login works behind Access.
- [ ] Modpack loads in the Technic Launcher (S11).
- [ ] Nightly database dump scheduled and the first one stored off the VM.

## Sources

- Technic Solder: Docker setup
- Technic Solder repository and wiki
- Cloudflare Access: app paths

- Cloudflare Access: policies and Bypass
- Cloudflare: self-hosted public applications